```
"""
Global Oil Production Monitor
Fetches and displays real-time oil production data by country using the EIA API.
Falls back to latest known data if the API is unavailable.
"""

import urllib.request
import json
import datetime

# ── Fallback data (barrels per day, approximate 2024 figures) ──────────────
FALLBACK_DATA = [
    {"country": "United States",  "region": "North America", "bpd": 20_000_000},
    {"country": "Saudi Arabia",   "region": "Middle East",   "bpd": 11_000_000},
    {"country": "Russia",         "region": "Europe/Asia",   "bpd": 10_500_000},
    {"country": "Canada",         "region": "North America", "bpd":  5_576_000},
    {"country": "Iraq",           "region": "Middle East",   "bpd":  4_520_000},
    {"country": "China",          "region": "Asia",          "bpd":  4_111_000},
    {"country": "UAE",            "region": "Middle East",   "bpd":  3_000_000},
    {"country": "Brazil",         "region": "South America", "bpd":  2_900_000},
    {"country": "Kuwait",         "region": "Middle East",   "bpd":  2_600_000},
    {"country": "Iran",           "region": "Middle East",   "bpd":  2_500_000},
    # Africa
    {"country": "Nigeria",        "region": "Africa",        "bpd":  1_700_000},
    {"country": "Angola",         "region": "Africa",        "bpd":  1_100_000},
    {"country": "Algeria",        "region": "Africa",        "bpd":    900_000},
    {"country": "Libya",          "region": "Africa",        "bpd":    700_000},
    {"country": "Egypt",          "region": "Africa",        "bpd":    600_000},
    {"country": "South Sudan",    "region": "Africa",        "bpd":    150_000},
    {"country": "Gabon",          "region": "Africa",        "bpd":    200_000},
    {"country": "Congo",          "region": "Africa",        "bpd":    250_000},
    {"country": "Equatorial Guinea","region":"Africa",        "bpd":    100_000},
]

REGION_EMOJIS = {
    "North America": "🌎",
    "Middle East":   "🌍",
    "Europe/Asia":   "🌏",
    "Asia":          "🌏",
    "South America": "🌎",
    "Africa":        "🌍",
}

BAR_WIDTH = 40  # characters
```

```
def format_bpd(n):
    """Format barrels per day as human-readable string."""
    if n >= 1_000_000:
        return f"{n/1_000_000:.2f}M bpd"
    elif n >= 1_000:
        return f"{n/1_000:.0f}K bpd"
    return f"{n:,} bpd"


def bar(value, max_value, width=BAR_WIDTH):
    """Render an ASCII progress bar."""
    filled = int(round(value / max_value * width))
    return "█" * filled + "░" * (width - filled)


def try_fetch_eia():
    """
    Attempt to fetch recent production data from the EIA open data API.
    Returns a list of dicts or None on failure.
    """
    # EIA open API — no key required for this endpoint
    url = (
        "https://api.eia.gov/v2/international/data/"
        "?frequency=annual&data[0]=value"
        "&facets[productId][]=53"        # crude oil production
        "&facets[unit][]=TBPD"
        "&sort[0][column]=period&sort[0][direction]=desc"
        "&length=20"
    )
    try:
        req = urllib.request.Request(url, headers={"User-Agent": "OilMonitor/1.0"
        with urllib.request.urlopen(req, timeout=8) as resp:
            payload = json.loads(resp.read())
            rows = payload.get("response", {}).get("data", [])
            if rows:
                return rows
    except Exception:
        pass
    return None


def display_table(data, title, use_live=False):
    """Pretty-print the production table."""
    now = datetime.datetime.utcnow().strftime("%Y-%m-%d %H:%M UTC")
    source = "Live EIA API" if use_live else "Latest verified data (2024)"
```

```
print()
print("=" * 70)
print(f"  🛢️  GLOBAL OIL PRODUCTION MONITOR")
print(f"  {title}")
print(f"  Updated: {now}  |  Source: {source}")
print("=" * 70)

# Group by region
regions = {}
for row in data:
    r = row["region"]
    regions.setdefault(r, []).append(row)

max_bpd = max(d["bpd"] for d in data)
total_world = sum(d["bpd"] for d in data)

rank = 1
for region, rows in sorted(regions.items()):
    emoji = REGION_EMOJIS.get(region, "🌐")
    print(f"\n  {emoji}  {region.upper()}")
    print(f"  {'─' * 66}")
    for row in sorted(rows, key=lambda x: x["bpd"], reverse=True):
        pct = row["bpd"] / total_world * 100
        b = bar(row["bpd"], max_bpd)
        print(f"  #{rank:<3} {row['country']:<22} {format_bpd(row['bpd']):>12
        print(f"       {b}")
        rank += 1

print()
print(f"  {'─' * 66}")
print(f"  🌐  WORLD TOTAL (shown countries): {format_bpd(total_world)}")
print("=" * 70)

# Africa summary
africa = [d for d in data if d["region"] == "Africa"]
if africa:
    africa_total = sum(d["bpd"] for d in africa)
    print(f"\n  🌍  AFRICA SUMMARY")
    print(f"  {'─' * 66}")
    print(f"  Total African production : {format_bpd(africa_total)}")
    print(f"  Share of shown world output: {africa_total/total_world*100:.1f}
    print(f"  Largest African producer : {sorted(africa, key=lambda x: x['bpd
    print("=" * 70)
print()
```

```
def main():
    print("\n 🔄  Attempting to fetch live data from EIA…")
    live = try_fetch_eia()

    if live:
        # Parse EIA response into our format
        parsed = []
        for row in live:
            parsed.append({
                "country": row.get("countryName", "Unknown"),
                "region":  row.get("countryRegionName", "Other"),
                "bpd":     int(float(row.get("value", 0)) * 1000),
            })
        display_table(parsed, "Top Oil Producers", use_live=True)
    else:
        print("  ⚠️  Live API unavailable — using latest verified data.\n")
        display_table(FALLBACK_DATA, "Top Oil Producers incl. Africa", use_live=F


if __name__ == "__main__":
    main()
```